Binary Search Logic (Perfect Square)

We use **binary search on the answer space** to check whether a number is a perfect square.

Core Logic

```
low = 1
high = num

while low <= high:
    mid = low + (high - low) // 2
    sq = mid × mid

    if sq == num:
        return true
    else if sq < num:
        low = mid + 1
    else:
        high = mid - 1

return false
```

Invariant: At every step, all possible answers lie inside [low, high]. After checking mid, it is permanently removed from the search space.

Dry Run 1: num = 15

Step	low	high	mid	mid²	Action
1	1	15	8	64	mid² > 15 → high = 7
2	1	7	4	16	mid² > 15 → high = 3
3	1	3	2	4	mid² < 15 → low = 3
4	3	3	3	9	mid² < 15 → low = 4

Stop: low = 4, high = 3 → **return false** ❌

Dry Run 2: num = 16

Step	low	high	mid	mid²	Action
1	1	16	8	64	mid² > 16 → high = 7

2	1	7	4	16	mid² == 16 → return true

✅ **Perfect square**

Dry Run 3: num = 17

Step	low	high	mid	mid²	Action
1	1	17	9	81	mid² > 17 → high = 8
2	1	8	4	16	mid² < 17 → low = 5
3	5	8	6	36	mid² > 17 → high = 5
4	5	5	5	25	mid² > 17 → high = 4

Stop: low = 5, high = 4 → **return false** ❌

Key Rules to Remember

- Always update bounds using mid ± 1
- Never allow the same mid to repeat
- Loop stops when low > high
- Binary search eliminates **half** the space every step

This is the correct and reusable binary search pattern for perfect square problems.